

HASHDIT

HashDit

Smart Contract Security

Venus CollateralGateway & Hub Precision

Security Audit Report

Venus

28 Sep 2026 – 29 Sep 2026

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Period	28 Sep 2026 - 29 Sep 2026
Overall Risk	Low
Project Name	Venus CollateralGateway & Hub Precision
Github	https://github.com/VenusProtocol/venus-periphery/pull/74 https://github.com/VenusProtocol/venus-protocol/pull/710 https://github.com/VenusProtocol/venus-protocol/pull/714

Smart Contract List

No.	Contract Name	Verdict	Details
1	CollateralGateway.sol (venus-periphery, PR#74)	Low	[L01]-Ack [L02]-Fixed [I01]-Ack
2	MarketFacet.sol (venus-protocol, PR#710)	Pass	—
3	PrimeV2.sol (venus-protocol, PR#710)	Pass	—
4	PrimeLens.sol (venus-protocol, PR#710)	Pass	—
5	ComptrollerLens.sol (venus-protocol, PR#714)	Informational	[I02]-Ack

Findings

[L01] Flash-loan collateral migration requires ~2x the migrated amount in source-market cash	
Contract	CollateralGateway.sol (venus-periphery PR#74)
Severity	Low
Description	When supplyFromCollateral migrates a position that backs a borrow, it takes a Core flash loan: it borrows the position's worth of the underlying (flashAmount) from the **source market vToken**; inside the executeOperation callback it first deposits that borrowed underlying into the Hub to mint the replacement vh collateral (so the underlying leaves that vToken), then redeems the old position from the **same** vToken and repays.

JavaScript

```
  // CollateralGateway.sol — executeOperation
  // (1) mint the replacement vh collateral with the
flash-borrowed amount (it leaves the source vToken)
  (_migrationShares, _migrationVTokens) = _depositAndSupply(
      _migrationHub, asset, _migrationVhMarket, amounts[0],
_migrationMinShares, user
  );
  // (2) redeem the old position from the SAME vToken, whose
physical cash is now reduced by flashAmount
  uint256 proceeds = _redeemToUnderlying(vToken,
_migrationVTokenAmount, asset, user);
```

When the flash loan is disbursed, transferOutUnderlyingFlashLoan has already performed doTransferOut, so the source vToken's physical balance getCashPrior() is reduced by flashAmount. The redeem inside the callback (redeemFresh) checks that same physical balance for liquidity:

JavaScript

```
  // VToken.sol — redeemFresh
  if (getCashPrior() < vars.redeemAmount) {
      ...TOKEN_INSUFFICIENT_CASH...
  }
```

The key point is that the vToken does track the outstanding loan in flashLoanAmount, but it adds that back **only for interest-rate calculations** (via the internal _getCashPriorWithFlashLoan()); the redeem liquidity check does not credit it, and the public getCash() equals getCashPrior() and does not add it back either:

JavaScript

```
  // VToken.sol
  function getCash() external view override returns (uint) {
      return getCashPrior();                 // raw physical
balance, does NOT add flashLoanAmount
  }
  function _getCashPriorWithFlashLoan() internal view returns
(uint) {
      return getCashPrior() + flashLoanAmount; // used only by
supplyRate / borrowRate
  }
```

So during the callback the source market must hold roughly flashAmount + redeemAmount — about 2x the migrated amount — in free cash, or the redeem reverts on getCashPrior() < redeemAmount.

Impact:
A migration that is large relative to the source market's available cash reverts, even when that cash exceeds the migrated amount (it must be ~2x). The whole call reverts atomically and the caller keeps their position, so there is **no loss of funds**; the impact is a non-obvious liveness/usability limit on large migrations, triggered under the specific condition that the source market's free cash is below 2x.

Recommendation

Size migrations against the source market's available cash rather than the position size: cap the migrated amount at roughly 50% of IVToken(vToken).getCash(), or split a large migration into chunks each within that bound (the market's cash is restored between chunks as each flash loan is repaid). Optionally add a pre-flight check that reverts early with a clear reason instead of failing deep inside redeemFresh:

JavaScript

```
uint256 redeemable = (vTokenAmount *
IVToken(vToken).exchangeRateStored()) / EXP_SCALE;
if (IVToken(vToken).getCash() < flashAmount + redeemable)
revert InsufficientSourceLiquidity();
```

Status

Acknowledged. The project confirmed the mechanism (the flash-borrowed amount leaves the source market before the in-callback redeem, and redeemFresh checks cash without counting the loan), and clarified that the ~2x requirement arises only when the Hub does not route the deposit back into the source market: the Hub's _routeDeposit sends the borrowed amount through its deposit queue, and when it lands in the source market's AdapterCoreV1 (vToken.mint), cash is restored before the redeem runs. Since the cash actually needed depends on the Hub's allocation at call time, a fixed pre-flight check would reject migrations that currently

	succeed, so no change was made; any shortfall reverts atomically with no fund risk.

[L02] withdrawPosition(type(uint256).max) reverts for a wallet-only position	
Contract	CollateralGateway.sol (venus-periphery PR#74)
Severity	Low

Description	withdrawPosition spends the caller's wallet Hub shares first and only frees the remainder from the vh market. It enters the Core (market) leg whenever walletShares < shares. For shares == type(uint256).max, walletShares is the caller's finite wallet balance, so the condition is always true and the Core leg always runs:
	
	JavaScript
	`uint256 walletShares = IERC20(hub).balanceOf(msg.sender);`
	`if (walletShares > shares) walletShares = shares;`
	`if (walletShares != 0)`
	`IERC20(hub).safeTransferFrom(msg.sender, address(this), walletShares);`
	
	`uint256 freedShares;`
	`if (walletShares < shares) freedShares = _freeFromMarket(hub, vhMarket, shares - walletShares);`
	
	When the caller holds Hub shares only in their wallet and has no receipts in the vh market, _freeFromMarket computes a zero redeem and reverts:
	
	JavaScript
	`uint256 balance = IVToken(vhMarket).balanceOf(msg.sender); // 0 for a wallet-only holder`
	`uint256 vTokens = balance; //`
	`0`
	`...`
	`uint256 errorCode = IVToken(vhMarket).redeemBehalf(msg.sender, vTokens); // redeem 0`
	`freedShares = IERC20(hub).balanceOf(address(this)) -`
	`balanceBefore; // 0`
	`if (freedShares == 0) revert NothingReceived();`
	
	If the caller has not granted the delegate (which a wallet-only withdraw should not need), redeemBehalf(msg.sender, 0) reverts even earlier: VBep20.redeemBehalf checks the delegate on its first line, before the amount matters. This contradicts the documented behavior: the interface states type(uint256).max "redeems the whole wallet balance and every vhMarket receipt", the delegate is only needed "if any of shares has to come

out of vhMarket", and "a market that cannot free the full amount pays out less rather than reverting".

```
JavaScript
// VBep20.sol - redeemBehalf
require(comptroller.approvedDelegates(redeemer, msg.sender),
"not an approved delegate");
```

Impact

A holder whose entire vh position sits in their wallet (for example, obtained by depositing into the Hub directly) cannot use the documented type(uint256).max / whole-position withdraw; any shares amount greater than their wallet balance reverts with a misleading error. Only passing the exact wallet balance works. No funds are lost (the transfer is rolled back).

Recommendation

Only enter the Core leg when the caller actually holds vh-market receipts, so a max/over-request on a wallet-only position redeems just the wallet shares as documented:

```
JavaScript
if (walletShares < shares &&
IVToken(vhMarket).balanceOf(msg.sender) != 0)
    freedShares = _freeFromMarket(hub, vhMarket, shares -
walletShares);
```

Alternatively, have _freeFromMarket return 0 when the caller's vh-market balance is 0 instead of calling redeemBehalf and reverting.

Status

Fixed in commit 6bffa62 and hardened in commit d0a5976. The Core leg now runs only when the caller holds vh-market receipts and has granted the delegate, so a wallet-only type(uint256).max withdraw pays out the wallet leg instead of reverting; the delegate check also stops dust receipts minted via the permissionless mintBehalf from forcing the Core leg on a caller who has not granted the delegate, and a call that frees nothing from either leg reverts with NothingReceived:

```
JavaScript
uint256 freedShares;
if (
    walletShares < shares &&
    IVToken(vhMarket).balanceOf(msg.sender) != 0 &&
    COMPTROLLER.approvedDelegates(msg.sender, address(this))
```

```
) {
    freedShares = _freeFromMarket(hub, vhMarket, shares -
walletShares);
}
if (walletShares + freedShares == 0) revert NothingReceived();
```

[I01] supplyFromCollateral / withdrawPosition require a pool-wide, call-outliving delegate grant	
Contract	CollateralGateway.sol (venus-periphery PR#74)
Severity	Informational

Description	supplyFromCollateral and withdrawPosition redeem from the caller's existing Core position through VToken.redeemBehalf, which requires the caller to have first granted the gateway a delegate via Comptroller.updateDelegate(gateway, true):
	
	JavaScript
	`// _redeemToUnderlying: burns the caller's receipts in place and pays this contract,`
	`// which requires a delegate grant from `from` rather than an ERC20 allowance`
	`uint256 errorCode = IVToken(vToken).redeemBehalf(from, vTokenAmount);`
	
	The updateDelegate grant is pool-wide: it authorizes the gateway to redeemBehalf and borrowBehalf against every Core position the caller holds, not only the market named in the call, and it persists after the transaction until the caller revokes it.
	
	Impact
	The gateway never operates on a third party — every behalf-call is bound to msg.sender — so the grant cannot be weaponized against another user through this contract. The residual consideration is a broad, standing authority that outlives the call: a user who does not revoke leaves the gateway able to redeem and borrow against their entire Core position for as long as the grant remains active.

Recommendation	Prompt users to revoke the grant with updateDelegate(gateway, false) after use, and document the pool-wide scope of the authorization. If the underlying protocol offers a narrower (per-market or single-use) delegation primitive, prefer it.

Status	Acknowledged. This is an architectural constraint: Core's updateDelegate is pool-wide and has no per-market or single-use form, so the gateway cannot request a narrower grant. The gateway only ever acts for msg.sender, and the interface docs and README already state the grant's scope and name updateDelegate(gateway, false) as the revoke.

[I02] Seize-precision fix changes seize amounts protocol-wide	
Contract	ComptrollerLens.sol (venus-protocol PR#714)
Severity	Informational

Description	PR714 changes how _calculateSeizeTokens rounds the amount of collateral to seize. The old code divided before multiplying: it collapsed the price and exchange-rate factors into an 18-decimal fixed-point ratio, flooring at each step, and only then multiplied that ratio by the repay amount, so precision lost while forming the ratio was carried into the result. The new code multiplies every factor into one integer and divides a single time:
	
	JavaScript
	`// ComptrollerLens.sol – _calculateSeizeTokens`
	`uint numerator = mul_(mul_(actualRepayAmount, liquidationIncentiveMantissa), priceBorrowedMantissa);`
	`uint denominator = mul_(priceCollateralMantissa, exchangeRateMantissa);`
	`seizeTokens = div_(numerator, denominator);`
	
	The fix is correct: the old formula systematically under-seized, and for a borrowed asset with many decimals — where the normalized price 10^(36 - decimals) is small — the intermediate ratio could floor all the way to zero, seizing no collateral. It is nonetheless a protocol-wide behavior change: every market now seizes the exact floor of the true value instead of an over-truncated, slightly smaller amount, and the new amount is always ≥ the old one.
	
	Impact:
	For assets reachable today (18 decimals or fewer) the difference is at most a few collateral base units — negligible in relative terms. The meaningful deviations, up to the zero-truncation above, require a borrowed asset with far more than 18 decimals; the 24-decimal Hub-share (vh) markets are the only such assets and are currently not borrowable (isBorrowAllowed = false), so that extreme is presently unreachable and the change is preventive there. The one live consideration is for liquidators that size the repay off-chain with the old formula exactly at the borrower's collateral balance: the new, marginally larger seize can exceed that balance and revert with LIQUIDATE_SEIZE_TOO_MUCH, requiring a slightly smaller repay. This is a compatibility note for liquidator integrations, not a defect.

Recommendation	No contract change is required — the fix itself is correct. Liquidator integrations should size the repay against the borrower's collateral balance using the new (exact-floor, marginally larger) seize amount, rather than the old formula, to avoid hitting LIQUIDATE_SEIZE_TOO_MUCH at the balance ceiling.
Status	Acknowledged. No contract change is required: the new formula multiplies once and divides once, so the seize amount is the exact floor of the true value, and the behavior change is the intent of the fix. Liquidators are expected to size the repay with the new seize amount to stay under LIQUIDATE_SEIZE_TOO_MUCH.